

# SOJOURN — MISSION OPERATIONS MANUAL

ARCHIVE COPY

INTERPLANETARY SURVEY AGENCY · DEEP SPACE FLIGHT OPERATIONS

**Document ISA-SOJ-OPS-014, Revision C** · Flight Software Build 1.0 Distribution: Flight Control Team, Sojourner Operations *Recovered from the Sojourner ground archive. Portions of this document were lost to media degradation and are marked accordingly.*

## ARCHIVIST'S NOTE

This is the only surviving copy of the Sojourner operations manual. Several pages, tables, and one full appendix could not be recovered. Where a figure or table is missing, the flight software itself remains the authority — read it directly. The probe still answers. That is what matters.

## 1. Mission Summary

Sojourner is a long-duration deep-space survey probe of the ISA Outer Horizons program. Launched decades before the loss of the program archive, it continues to return telemetry across a one-way signal delay now measured in tens of hours. Its scientific payload comprises a magnetometer, an inertial measurement unit, a thermal sensor suite, a bus power monitor, a radiation counter, a star tracker, and a narrow-field imaging camera.

The flight software source code did not survive. What survives is this manual, the ground station, and the probe. Mission continuation therefore proceeds the way the late-program flight team learned to work: by reading and, where necessary, **rewriting the probe's memory directly** over the uplink — powering down failing hardware, adjusting mission parameters, and installing corrections, verified only by what returns on the downlink.

⚠ Every uplinked change takes effect on a machine you cannot physically reach. There is no undo. A careless write can silence the probe. Read §7 (Fault Protection) before you begin.

## 2. Ground Station & Signal Path

The flight team communicates with Sojourner through the ground station console. Two streams share the link:

- **Downlink (telemetry):** the probe transmits a status frame at a fixed cadence. Each frame arrives on a line beginning `TLM`. See §5.
- **Uplink (commands):** the operator sends one command per line. The probe acknowledges each. See §3.

Because of signal delay and bandwidth limits, mission control works in a deliberate rhythm: observe the downlink, decide on a single change, uplink it, then wait for the telemetry that confirms — or refutes — the result. Plan each command before you send it.

### 3. Uplink Command Format

Every command is a single line of printable text:

```
VERB [arguments] *CCCC
```

CCCC is a **CRC-16/CCITT (0xFFFF init, polynomial 0x1021)** checksum, written as four hexadecimal digits, computed over **every** character that precedes the **\*** — including the separating space. The probe recomputes the checksum and rejects the command ( NAK E01 ) if it does not match. This guards against corruption on the long uplink path.

Numeric addresses and byte values are hexadecimal. A **0x** prefix is optional on addresses.

**Worked example.** To send PING , checksum the string PING (four letters and one trailing space): the CRC is 2F40 . The full line is:

```
``
PING *2F40
``
```

#### 3.1 COMMAND REFERENCE

| VERB | ARGUMENTS         | ACTION                                   | SUCCESS REPLY                                                 |
|------|-------------------|------------------------------------------|---------------------------------------------------------------|
| PING | —                 | Liveness check                           | ACK PING                                                      |
| PEEK | addr len          | Read len bytes (1–64, decimal) from addr | ACK PEEK <hex bytes>                                          |
| POKE | addr<br>b0[b1...] | Write hex bytes (up to 32) to addr       | ACK POKE <count>                                              |
| STAT | —                 | Report probe status                      | ACK STAT mode=... up=...s reboots=...<br>fault=... load=...mW |
| SAFE | —                 | Command safe mode (see §7.3)             | ACK SAFE                                                      |
| NOOP | —                 | No operation (accepted, does nothing)    | ACK NOOP                                                      |

#### 3.2 VERIFIED EXAMPLES

All checksums below are correct for Flight Software Build 1.0. Type them exactly.

```

PING *2F40                -> ACK PING
STAT *99A2                -> ACK STAT mode=1 up=45s reboots=0 fault=0
load=775mW
NOOP *6EFF                -> ACK NOOP
PEEK 0x00000000 4 *C9E8   -> ACK PEEK 00000220
POKE 0x20005000 2A *75A6  -> ACK POKE 1

```

Note the `PEEK 0x00000000 4` result: the four bytes read back as `00 00 02 20`, which is the value `0x20020000` stored **least significant byte first**. Sojourn's processor is little-endian; keep this in mind whenever you read or write multi-byte values.

### 3.3 ERROR REPLIES

A command that is not accepted returns `NAK` and a code:

| CODE | MEANING                                              |
|------|------------------------------------------------------|
| E01  | Bad checksum — the CRC did not match. Recompute it.  |
| E02  | Unknown verb.                                        |
| E03  | Address not mapped (see §4).                         |
| E04  | Protected region — the write was refused (see §4.2). |
| E05  | Malformed arguments or length out of range.          |
| E06  | Subsystem busy; retry shortly.                       |

An `ACK` confirms only that the probe **received and executed** the command. It does **not** confirm that the change had the effect you intended. For that, read the telemetry.

## 4. Memory Organization

Sojourn's flight computer presents a single address space. Only a broad map survived; the detailed layout table (ISA-SOJ-MEM-002) was lost.

### 4.1 REGIONS (COARSE)

| RANGE                   | CONTENTS                                                                         | WRITABLE? |
|-------------------------|----------------------------------------------------------------------------------|-----------|
| 0x00000000 – 0x0003FFFF | Boot firmware, recovery services, and the master program image (read-only store) | No        |
| 0x20000000 – 0x20000FFF | Reserved system region (recovery state, timers)                                  | No        |
| 0x20001000 –...         | <b>Flight program &amp; working memory</b> — the running software and its data   | Yes       |
| high memory             | Peripheral register interface; imaging frame store                               | Yes       |

TABLE 4-2 – SUBSYSTEM BASE ADDRESSES

PAGES MISSING

*\*The table giving the base address of each subsystem's register block – sensors, camera, and others – was not recovered. These addresses can be re-derived by examining the flight program (§8) or by careful PEEK survey of the peripheral region.\**

4.2 WRITE PROTECTION

The probe refuses ( NAK E04 ) any POKE to the boot/program store ( 0x00000000 – 0x0003FFFF ) and to the reserved system region ( 0x20000000 – 0x20000FFF ). This is deliberate: it is what allows the probe to recover itself after a fault (§7). You cannot damage these regions, so do not waste uplink attempts on them.

The **running flight program** in working memory (from 0x20001000 ) is **not** protected. Writes there take effect immediately on the live software. This is the mechanism by which the mission is maintained — and the mechanism by which it can be lost. See §7.

5. Downlink Telemetry Format

Each telemetry frame is transmitted as one line: the literal TLM , a space, then the frame encoded as hexadecimal. Decode the hex to bytes, then parse as below. All multi-byte fields are **big-endian** in the frame (note: this differs from memory, which is little-endian — the telemetry encoder byte-swaps for the downlink).

5.1 FRAME STRUCTURE

EB 90 | LEN | ---- payload (LEN bytes) ---- | CRC16

| FIELD    | SIZE | NOTES                                     |
|----------|------|-------------------------------------------|
| Sync     | 2    | Always 0xEB90 . Marks frame start.        |
| Length   | 1    | Payload length in bytes.                  |
| Payload  | LEN  | Fixed header followed by channels (§5.2). |
| Checksum | 2    | CRC-16/CCITT over Length + Payload.       |

5.2 PAYLOAD HEADER

The payload always begins with this fixed header:

| OFFSET | FIELD         | SIZE | MEANING                          |
|--------|---------------|------|----------------------------------|
| 0      | Frame counter | 2    | Increments each frame; wraps.    |
| 2      | Uptime        | 4    | Seconds since last (re)start.    |
| 6      | Mode          | 1    | 0 = BOOT, 1 = NOMINAL, 2 = SAFE. |

| OFFSET | FIELD       | SIZE | MEANING                              |
|--------|-------------|------|--------------------------------------|
| 7      | Reboots     | 1    | Lifetime restart count (see §7).     |
| 8      | Last fault  | 1    | Cause of most recent restart (§7.2). |
| 9      | Bus voltage | 2    | Millivolts.                          |
| 11     | Load        | 2    | Total power draw, milliwatts.        |
| 13     | Channels    | ...  | Zero or more data channels (§5.3).   |

### 5.3 DATA CHANNELS

After the header, the payload carries a sequence of **channels**, each:

ID | LEN | value (LEN bytes)

A channel is present **only when its subsystem is powered and being reported**. A powered-down or unreported sensor's channel is simply **absent** from the frame — it does not appear as zero. Watching a channel appear or disappear is the primary way to confirm that an uplink had the effect you intended.

| ID   | SUBSYSTEM           | VALUE    | INTERPRETATION                  |
|------|---------------------|----------|---------------------------------|
| 0x00 | Magnetometer (MAG)  | int32    | Field strength, nT              |
| 0x01 | Inertial unit (IMU) | int32    | Rotation rate ×100, deg/s       |
| 0x02 | Thermal (THM)       | int32    | Temperature ×10, °C             |
| 0x03 | Bus monitor (PWR)   | int32    | Bus voltage, mV                 |
| 0x04 | Radiation (RAD)     | int32    | Cumulative particle count       |
| 0x05 | Star tracker (STR)  | int32    | Attitude quaternion term ×10000 |
| 0x43 | Camera (CAM)        | 12 bytes | Capture metadata (§6.3)         |

In SAFE mode the probe transmits the header only, with no data channels, to conserve power.

### APPENDIX C — RESERVED / DIAGNOSTIC CHANNELS

PAGES MISSING

*\*Flight crews reported occasional channels in the downlink not listed in Table 5-3 above. The appendix cataloguing them was not recovered. If you observe a channel ID you do not recognize, decode it by its LEN and study its value across frames.\**

# 6. Payload Subsystems

## 6.1 SCIENCE SENSORS

Sojourn carries six sensor subsystems. Each is identified by a **slot number** matching its telemetry channel ID:

| SLOT | ID  | SENSOR                    |
|------|-----|---------------------------|
| 0    | MAG | Magnetometer              |
| 1    | IMU | Inertial measurement unit |
| 2    | THM | Thermal sensor suite      |
| 3    | PWR | Bus power monitor         |
| 4    | RAD | Radiation counter         |
| 5    | STR | Star tracker              |
| 6    | CAM | Imaging camera (§6.2)     |

Each sensor is governed by a small block of registers. The **layout within a slot** survived; the **base address of the register block did not** (see the missing Table 4-2). Each slot occupies 16 bytes:

| OFFSET IN SLOT | REGISTER | MEANING                                                       |
|----------------|----------|---------------------------------------------------------------|
| +0x00          | CONTROL  | Bit 0 = POWER (1 = powered on). Other bits reserved.          |
| +0x04          | STATUS   | Bit 0 = READY (data valid). Bit 1 = FAULT (degraded/failing). |
| +0x08          | DATA     | Latest reading (signed, sensor-specific units per §5.3).      |
| +0x0C          | POWER    | Present draw, milliwatts (0 when unpowered).                  |

Slots are laid out consecutively, 16 bytes apart, in slot-number order. Clearing a sensor's POWER bit removes it from the power budget and, in turn, from the downlink. (Note that the flight program also maintains an internal list of which sensors it actively reports; that list is part of the working program, not these registers.)

⚠ Powering a sensor off is a memory write to the live probe. Confirm the **slot base address** before you send it — writing the right bit to the wrong address can corrupt the flight program.

## 6.2 IMAGING CAMERA

The camera captures a 64×64, 8-bit grayscale frame of a commanded target. Its control registers (base address in the missing Table 4-2) are:

| OFFSET | REGISTER | MEANING                                                                             |
|--------|----------|-------------------------------------------------------------------------------------|
| +0x00  | CCTRL    | Bit 0 = CAPTURE (take one frame now; self-clears). Bit 1 = AUTO (periodic capture). |
| +0x04  | CSTAT    | Bit 0 = READY. Bit 1 = BUSY. Bit 2 = POINT_ERR (no attitude reference).             |

| OFFSET | REGISTER       | MEANING                                               |
|--------|----------------|-------------------------------------------------------|
| +0x08  | TARGET         | Target selector (index into the onboard target list). |
| +0x0C  | EXPOSURE       | Exposure time, milliseconds (1–10000).                |
| +0x10  | GAIN / BINNING | Analog gain (16-bit) and pixel binning (16-bit).      |
| +0x14  | FRAME_ID       | Increments on each completed capture.                 |
| +0x18  | FRAME_ADDR     | Address of the most recent frame in memory.           |
| +0x1C  | FRAME_LEN      | Frame size in bytes.                                  |
| +0x20  | HIST_MEAN      | Mean pixel brightness of the last frame.              |
| +0x24  | HIST_MAX       | Peak pixel brightness.                                |
| +0x28  | SAT_PCT        | Percentage of saturated (white-clipped) pixels.       |
| +0x2C  | STARS          | Count of resolved point sources.                      |

**Attitude dependency.** A capture requires an attitude reference: the star tracker (slot 5) must be powered and READY, or the camera raises POINT\_ERR in CSTAT and takes no frame. Do not power down the star tracker while imaging objectives are active.

**Exposure.** Too long an exposure saturates the sensor: SAT\_PCT rises toward 100 and STARS falls toward zero as point sources bleed into one another. Too short an exposure yields a dim frame with few resolved stars. A well-exposed frame maximizes STARS at low SAT\_PCT. These statistics are reported in the downlink (§6.3), so exposure can be tuned from telemetry alone.

TABLE 6-4 — ONBOARD TARGET LIST

PAGES MISSING

*\*The TARGET register selects an entry from a list of survey targets stored in the flight program's configuration data. The catalogue of targets, and the location and format of that list, were not recovered. Locate it within the flight program (§8) to retarget the camera.\**

6.3 CAMERA TELEMETRY (CHANNEL 0X43)

After at least one capture, the camera reports a 12-byte channel each frame. All fields are 16-bit, big-endian:

| OFFSET | FIELD             |
|--------|-------------------|
| 0      | Frame ID          |
| 2      | Target index      |
| 4      | Exposure (ms)     |
| 6      | Mean brightness   |
| 8      | Saturated percent |

| OFFSET | FIELD               |
|--------|---------------------|
| 10     | Resolved star count |

**Recovering an image.** The pixel data is never downlinked in telemetry — only these statistics. To retrieve an actual frame, read the frame store directly with `PEEK`, using the address in `FRAME_ADDR` and the size in `FRAME_LEN` (4096 bytes), 64 bytes per command. Reassemble the bytes on the ground into a 64×64 image. This is slow and deliberate; budget your uplink accordingly.

## 7. Fault Protection & Recovery

Sojourn is designed to survive operator error and hardware upset. Read this section before writing to working memory.

### 7.1 THE WATCHDOG

An independent timer in the protected system region must be serviced regularly by healthy flight software. If the software stops servicing it — because it has crashed, hung, or been corrupted by a bad uplink — the timer expires (after approximately three seconds) and the probe **restarts itself**.

### 7.2 RESTART & IMAGE RECOVERY

On any restart, the boot firmware **reloads the entire flight program from the protected master image** before running it. This means:

- Any change you wrote to working memory is **erased** on restart. The probe returns to its original, as-built behavior.
- The **reboot counter** in telemetry increments, and **uptime** resets toward zero — your signal that a restart occurred.
- The **last-fault** field reports the cause:

| VALUE | MEANING                                                                   |
|-------|---------------------------------------------------------------------------|
| 0     | None (clean start).                                                       |
| 1     | Watchdog timeout — software stopped responding.                           |
| 2     | Processor fault — an illegal operation (e.g. execution of a bad address). |
| 3     | Image integrity failure.                                                  |

Recovery is automatic and requires no action from the ground. If you brick the probe with a bad write, wait for the reboot; it will come back. You will, however, lose every in-memory change you had made — so keep a record of your commands and be ready to re-send the good ones. (The ground station's command history exists for exactly this reason.)

### 7.3 SAFE MODE

In SAFE mode the probe powers down non-essential subsystems and reduces telemetry to the header only. It may be entered automatically after a sustained sensor fault, or commanded with `SAFE`. Safe mode is a low-power holding



state, useful when you need the probe stable while you plan.

## 8. Working With the Flight Program

Several tasks in this manual — locating a subsystem's register base (Table 4-2), finding the target list (Table 6-4), correcting a behavior in the running software — require understanding the flight program itself. The program image is available for study (it is the read-only store described in §4). Standard practice on the late-program flight team was to load the image into a disassembler, identify the relevant data structures and code, and derive the exact addresses to `PEEK` and `POKE` from there.

This is unavoidable. The detailed memory and configuration tables that would otherwise give these addresses directly are among the pages this archive lost. The probe, however, has not changed. Everything you need to know about it can be learned by reading it.

### CHANGE HISTORY

Rev A — initial issue, pre-launch.

Rev B — updated telemetry channel table; added imaging payload.

Rev C — *recovered fragment; change list illegible*

*End of recovered document. Sections and appendices not listed in this copy did not survive. When the manual and the probe disagree, the probe is correct.*